



Advertisement: Support JavaWorld, click here!

October 2000

HOME

FEATURED
TUTORIALS

COLUMNS

NEWS &
REVIEWS

FORUM

JW
RESOURCES

ABOUT
JW

Validation with Java and XML Schema, Part 2

Using XML Schema for constraining Java data

Summary

Constraining XML by using DTDs or XML Schema is a well-accepted practice. However, using those same mechanisms for constraining Java method arguments is uncharted territory. In this article, Brett looks at a roadmap for taking Java method parameters and validating them against constraints in an XML document. Various approaches will be examined, and you will begin to actually code the utilities for converting those XML constraints into usable Java utilities. *(3,000 words)*

By Brett McLaughlin

With the wealth of application development in Java today, there seems to be an API for almost everything: remote method invocation (RMI), reusable business components (EJB), manipulating XML (SAX, DOM, JDOM, JAXP), and user interfaces (Swing) as well as writing a help system (JavaHelp). Yet programmers still spend hours and even days on each project, working out validation routines. Mind you, those aren't complex business formulas but ensuring that a value is of the correct data type when submitted via an HTML form or checking the range of a shoe size. Somehow, with all the recent focus on enterprise applications, some of a programmer's core tasks have been overlooked.

Read the whole "Validation with Java and XML Schema" series:

- [Part 1. Learn the value of data validation and why pure Java isn't the complete solution for handling it](#)
- [Part 2. Use XML Schema for constraining Java data](#)
- [Part 3. Parsing XML Schema to validate data](#)
- [Part 4. Build Java representations of schema constraints and apply them to Java data](#)

In an effort to resolve that problem, at least until the powers that be come up with a robust API for validation, this series takes a detailed look at validation in Java. That isn't an explanation on using JavaScript in your HTML or expensive third-party libraries but instead on creating a simple validation framework based on existing standards. The focus is on ease of use and a simple means to add new validation rules into the data constraints without cluttering business and presentation logic with validation details.

The story so far

To get started, you should take the time to read [Part 1](#) in the series. In that article, I looked at several existing options for validation, particularly pure Java options. Both inline validation (such as directly in a servlet or Enterprise JavaBean) as well as helper classes (such as Jason Hunter's `ParameterParser` class) often still resulted in code that was cluttered and that mixed validation with business and application logic. Additionally, you were left to deal with numerous `try/catch` blocks and throwing exceptions. It also left the unwanted problem of having to constantly recompile, even for the most minor changes in data constraints (such as changing an allowed range from between 0 and 20 to between 1 and 20).

I also discussed Java property files as a way to handle that problem. First, a small clarification: while Java does *allow* property files to have multiple period separated keys (`key1.key2.key3 = value`), it does not allow their use in any meaningful way. For example:

```
ldap.hostname = galadriel.middleearth.com
ldap.port = 389
ldap.userDN = cn=Directory Manager
ldap.password = foobar
```

It would seem that that sample entry in a Java properties file represents a logical grouping; all the entries start with the `ldap` key. However, that is not the case with standard Java APIs. That entry set is functionally equivalent to:

```
hostname = galadriel.middleearth.com
port = 389
userDN = cn=Directory Manager
password = foobar
```

In other words, there is no means to get, for example, all the keys with an `ldap` root. That makes using multiple-key values useful for human readability only and essentially a waste of time in actual application programming without custom or third-party libraries. So Java property files, too, are not suitable for large validation rules.

Finally, I briefly explained using XML to store validation constraints. XML Schema was addressed specifically, as it already has a mechanism to constrain an XML document that is type-safe and verbose. It allows range setting, specification of an enumeration of acceptable values, and a simple syntax. In this article, I'll delve deeper into using XML for constraining data in your Java applications, and you'll begin to write some code to put XML to work. First, I'll address your options within the XML realm.

Perusing the options

So now you know you want to use XML Schema for data constraints. So let's start talking specifics. The biggest issue you need to address is that your constraints are in one language, XML, and your data is in another, Java. So some sort of conversion must take place. Should you convert all of your data to XML, and then validate that XML against your schema? Should your XML Schema somehow be converted to Java, and those objects used to validate your data? What's the right thing to do here? Is it some mixture of the two?

Java to XML

The first option, converting Java data to XML, is actually fairly simple. With the rise in popularity of XML Data Binding, there are several frameworks available that convert, or marshal, a Java object

to an XML document. One of those, which I wrote (yes, it's a shameless plug!), is discussed in detail in "Objects, Objects Everywhere" (see [Resources](#)). A complete working package is provided for converting between Java and XML. The API is simple, lightweight, and intuitive -- all desirable qualities for your validation solution.

However, data binding is not as perfect as it may seem when you look a little deeper. First, your Java data may often come in four, five, or even more different pieces. Imagine a form that receives 15 input fields, all as separate Java objects (Strings, in this case). Each would have to be assembled into a single object, marshaled to XML, and then validated against the XML Schema. Your code, then, has to include logic for converting multiple objects into one object suitable for conversion to XML. So your once-simple solution is already getting convoluted.

Further, that option does not allow for any optimization. You can't store the XML Schema in memory in your JVM, and the only real advantage you might introduce is caching the actual XML Schema document (as a DOM or JDOM Document object, perhaps). In other words, there is no performance gain over multiple validation calls. While that might seem like icing on the cake, consider that validation, especially of form data, happens hundreds and even thousands of times per page, per day (or hour, or minute!). Caching, or some sort of performance gain, should really be expected over multiple invocations of the validation. Additionally, parsing XML is a costly operation, and even if the XML Schema document is cached, the marshalled Java object, resulting in an XML document, must be parsed at each validation call. Thus, the conversion from Java to XML doesn't seem to be such a good idea.

XML to Java

Since conversion from Java to XML doesn't seem to be a good idea, let's take a look at the flipside: converting XML to Java. In that case, your Java data would stay as is and would not need to be marshaled into XML. You would instead need to convert your XML Schema constraints into Java objects. Those objects can then take in data and return a result, indicating if the data was valid for the constraints that the object and the underlying XML Schema represented. That is a much more natural case for Java developers as well, as it allows them to stay in a Java environment.

Another advantage to that technique is that it effectively isolates XML Schema from the equation. Using schemas, then, becomes a decision tied only to the conversion from XML to Java, and not the use of the resultant Java objects at all. In other words, if the implementation of those validation classes was changed to convert an XML document (not a schema) to a validation object, the developer would still see the same interface for validation; no application code would need to change. Why is that a big deal? Well, there are two reasons. First, XML Schema is still being finalized, and minor changes may occur. Using that design ensures that you can code to the validation classes covered in that series and, even if XML Schema's specification changes and the implementation of the classes changes, your application code stays the same. Second, there is still some widespread concern over the acceptance of XML Schemas. If they did not satisfy your needs, or if they perhaps were overcomplicated for your application, you could switch to a simpler mechanism (such as simple XML documents or Relax) and still have the same code routines work.

Going back to some original concerns, that also means that your XML Schema document only has to be parsed a single time. The schema is converted to Java objects, representing constraints, and then stored in memory. Data can be validated against the objects over and over without any additional parsing ever occurring. That addresses some of the performance issues I discussed in the section on converting from Java to XML. That is even more critical when the XML Schema might be located across a network, requiring network transfer time for each parsing.

So it seems clear that conversion from XML to Java is the right way to go. Additionally, you want more than just a simple object in Java (such as one that might be produced by unmarshaling an XML Schema document to Java, as in data binding); it should take in a piece of data, and then return whether the data is valid for the constraints supplied in the XML Schema.

The game plan

With those basic design decisions in place, it's time to start outlining your classes and decide what they will look like.

Elements and attributes

The first order of business is to decide what sort of XML Schema constructs you need to support. While it might seem logical to try to support everything in the XML Schema specification, that is both an enormous task (certainly more articles than this series can bear!) as well as counterproductive. For example, the `minOccurs` and `maxOccurs` attributes, a core part of XML Schema, have no meaning in the context of validation; a value either exists or does not, and cannot appear repeatedly in that context. So already you can see that some schema constructs are not needed for your validation code.

In fact, looking back at Part 1, XML Schema elements are completely unnecessary. Remember that elements in XML are usually meant to represent repeatable, complex data structures. Most often, they are mapped to Java objects (nonprimitive ones, mostly) rather than single data values. For example, examine this XML Schema:

```
<?xml version="1.0"?>

<schema targetNamespace="http://www.enhydra.org"
        xmlns="http://www.w3.org/1999/XMLSchema"
        xmlns:enhydra="http://www.enhydra.org"
>

    <complexType name="ServiceConfiguration">
        <attribute name="name" type="string" />
        <attribute name="version" type="float" />
    </complexType>

    <element name="serviceConfiguration" type="ServiceConfiguration" />

</schema>
```

That schema could easily be mapped to the Java object seen here:

```
public class ServiceConfiguration {
    private String name;
    private float version;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public float getVersion() {
        return version;
    }
}
```

```

    public void setVersion(float version) {
        this.version = version;
    }
}

```

Here, the attributes in the schema map to Java primitives (the types of data you will be validating), while the elements map to complex Java objects. What does that mean to you? Actually, quite a bit. You can essentially dispense with the support for elements in an XML Schema. Instead, focusing on the attribute type can make your job both simpler and more manageable.

In your schemas, then, you need to supply a name for the data for each data member you want to validate. That name acts as more of a data *identifier* than an instance variable name. That means that passing your validation framework a piece of data with the name supplied will result in the data being validated against that identifier's constraints. You can then specify the constraints on the type, such as the data type (int, String, and so forth) and allowed values.

With those decisions starting to firm up your game plan, let's look back at the XML Schema discussed in Part 1.

```

<?xml version="1.0"?>

<schema targetNamespace="http://www.buyShoes.com"
        xmlns="http://www.w3.org/1999/XMLSchema"
        xmlns:buyShoes="http://www.buyShoes.com"
>

    <attribute name="shoeSize">
        <simpleType baseType="integer">
            <minExclusive value="0" />
            <maxInclusive value="20" />
        </simpleType>
    </attribute>

    <attribute name="brand">
        <simpleType baseType="string">
            <enumeration value="Nike" />
            <enumeration value="Adidas" />
            <enumeration value="Dr. Marten" />
            <enumeration value="V-Form" />
            <enumeration value="Mission" />
        </simpleType>
    </attribute>

</schema>

```

Here, several things are happening. Two data identifiers are setup: "shoeSize" and "brand." The shoe size must be a Java int, greater than 0, and less than or equal to 20. The brand is a Java String, and several allowed values are specified. I'll discuss each of those constraints in a little more detail now.

Data types

Supporting data type validation is fairly easy. You'll start the framework with basic Java primitives.

In that aspect, your validation code (at least that aspect of it) is similar to the `ParameterParser` class I covered in Part 1. A Java `String` is supplied to each method you will code. That is to account for almost all input, especially from an HTML form, being in that format. If that is new ground for you, ask your fellow developers with experience; data almost always comes in as simple Java `Strings`. The result of the conversion should, of course, be an object of the correct data type:

```
public int getIntParameter(String value)
    throws NumberFormatException {
    return Integer.parseInt(value);
}
```

That is just a general code fragment; in the next section, you'll start to code actual validation classes. However, you should see that the data type conversion is pretty easy to ensure.

Value constraints

The other major facet you want to support is *value constraints*. Value constraints are items that restrict the allowed values once data type has been established, such as the shoe size restriction that ensures it is between 1 and 20 (inclusive). Those constraint types are generally detailed by two XML Schema constructs: the enumeration, which specifies allowed character values, and the `minXXX` and `maxXXX` keywords, which specify minimum and maximum allowed values (both inclusively and exclusively).

By supporting those two keyword sets, both nested within the `simpleType` schema feature, you can allow representation of almost all basic data constraints. You should also notice that you've narrowed down the keywords you have to support even further. In a later article, I'll touch on the pattern keyword, allowing pattern matching (like regular expressions) within character values. That will add yet another constraint tool to your growing arsenal.

So now that you've made it through the design phase (all the talk!), you can get down to the code. Let's look at starting work on a validation framework.

Getting down to it

I've written about enough -- let's get down to the code. You basically have four classes that you need to code:

- The `Constraint` class, which represents constraints for a single type, like the `shoeSize` type.
- The `Validator` class, which provides an interface for allowing developers to pass in data, and find out if the data is valid.
- The `SchemaParser` class, which parses an XML Schema and creates the `Constraint` objects for use by the `Validator` class.
- The `DataConverter` helper class, which will convert from XML Schema data types to Java data types and perform other data type conversions for you.

As that code is being developed for this article and the Enhydra Application Server (which you can check out in [Resources](#)), all code is in the `org.enhydra.validation` package. In addition, the code in this series is open source, meaning you can change whatever you like (such as adding

features and functionality). You are also welcome to email those changes to the Enhydra mailing list or to me, and they will be added to the main code base. So with all the details set, let's get into classes.

The Constraint class

The first class is perhaps the simplest to actually turn from design to compilable code. The Constraint class will represent the various constraints for a specific data type. It is identified by an identifier, not surprisingly! Thus, you can easily code the constructor to accept that as a required parameter. Then you need methods for the various types of constraints such as the minimum inclusive value. An accessor ("get" methods), mutator ("set" methods), and interrogator ("has" methods) are needed for each type of constraint. So, for the minInclusive constraint, you would have the following three methods:

```

/**
 * <p>
 * This will set the minimum allowed value for this data type
(inclusive).
 * </p>
 *
 * @param minInclusive minimum allowed value (inclusive)
 */
public void setMinInclusive(double minInclusive) {
    this.minInclusive = minInclusive;
}

/**
 * <p>
 * This will return the minimum allowed value for this data type
(inclusive).
 * </p>
 *
 * @return <code>double</code> - minimum value allowed (inclusive)
 */
public double getMinInclusive() {
    return minInclusive;
}

/**
 * <p>
 * This will return <code>true</code> if a minimum value (inclusive)
constraint
 * exists.
 * </p>
 *
 * @return <code>boolean</code> - whether there is a constraint for the
 * minimum value (inclusive)
 */
public boolean hasMinInclusive() {
    return (minInclusive != Double.NaN);
}

```

Similar methods are provided for the minExclusive, maxInclusive, and maxExclusive constraints. Additionally, you need a means of adding allowed values, used when enumerations

are supplied in the XML Schema. You also need similar methods for returning the allowed values and seeing if any allowed values exist.

```

/**
 * <p>
 * This will add another value to the list of allowed values for this
data type.
 * </p>
 *
 * @param value <code>String</code> value to add.
 */
public void addAllowedValue(String value) {
    allowedValues.add(value);
}

/**
 * <p>
 * This will return the list of allowed values for this data type.
 * </p>
 *
 * @return <code>List</code> - allowed values for this
<code>Constraint</code>.
 */
public List getAllowedValues() {
    return allowedValues;
}

/**
 * <p>
 * This checks to see if there are only a certain set of allowed
values.
 * </p>
 *
 * @return <code>boolean</code> - whether there are allowed values for
this type.
 */
public boolean hasAllowedValues() {
    if (allowedValues.size() == 0) {
        return true;
    } else {
        return false;
    }
}

```

And, of course, there must be a means to set the data type and retrieve it, according to the schema. That is the Java equivalent of the type specified by the schema type or baseType attribute. Similar methods are provided for that functionality:

```

/**
 * <p>
 * This will allow the data type for the constraint to be set. The type
is specified

```

```

    *    as a Java String.
    * </p>
    *
    * @param dataType String this is the Java data type for
this constraint.
    */
    public void setDataType(String dataType) {
        this.dataType = dataType;
    }

    /**
    * <p>
    * This will return the String version of the Java data
type for this
    * constraint.
    * </p>
    *
    * @return String - the data type for this constraint.
    */
    public String getDataType() {
        return dataType;
    }

```

And as simply as that, you are finished with the Constraint class. You can view the complete class in [Resources](#). If it seems as if that class is just a bunch of get and set methods, you are exactly right! That is just a Java representation of a set of data constraints. The next class you'll code though, the Validator class, will use it heavily.

The Validator class

The Validator class plays the most visible role in your validation framework -- it is the means for developers to interact with validation. Developers get an instance of the class and pass in data to be validated, getting a simple boolean result. They can then refuse the data, throw errors, or take other courses of action.

First, though, a word about how that class is set up. Unlike most classes, the Validator class is not best constructed directly (using the new keyword). The same servlet, running in multiple threads, multiple servlets in multiple threads, and multiple classes in multiple threads, may all use the validation code. If each object instantiated a new Validator instance, parsing would end up taking place, often for the same schema, many times. Instead, you want parsing of an XML Schema to take place only once. Because of that, you employ the Singleton design pattern, which ensures that only one instance of a given class is made available to all threads in the JVM. However, you make a slight modification. Because you don't need just one instance, but one instance *per XML Schema*, you will actually use a number of instances, each one being tied to a specific schema. Requests for an instance for a schema in which the instance already exists result in that existing instance being returned and parsing not reoccurring. So you can now create the core of the Validator class.

```

package org.enhydra.validation;

import java.net.URL;
import java.util.HashMap;
import java.util.Map;

```

```

/**
 * <p>
 * The <code>Validator</code> class allows an application component or
 * client to
 * provide data, and determine if the data is valid for the requested
 * type.
 * </p>
 */
public class Validator {

    /** The instances of this class for use (singleton design pattern) */
    private static Map instances = null;

    /** The URL of the XML Schema for this <code>Validator</code> */
    private URL schemaURL;

    /** The constraints for this XML Schema */
    private Map constraints;

    /**
     * <p>
     * This constructor is private so this the class cannot be instantiated
     * directly, but instead only through <code>{@link #getInstance()}
     </code>.
     * </p>
     */
    private Validator(URL schemaURL) {
        this.schemaURL = schemaURL;
        constraints = new HashMap();

        // parse the XML Schema and create the constraints
    }

    /**
     * <p>
     * This will return the instance for the specific XML Schema URL. If a
     schema
     * exists, it is returned (as parsing will already be done);
     otherwise,
     * a new instance is created, and then returned.
     * </p>
     *
     * @param schemaURL <code>URL</code> of schema to validate against.
     * @return <code>Validator</code> - the instance, ready to use.
     */
    public static Validator getInstance(URL schemaURL) {
        if (instances != null) {
            if (instances.containsKey(schemaURL.toString())) {
                return (Validator)instances.get(schemaURL.toString());
            } else {
                Validator validator = new Validator(schemaURL);
                instances.put(schemaURL.toString(), validator);
                return validator;
            }
        }
    }
}

```

```

        } else {
            instances = new HashMap();
            Validator validator = new Validator(schemaURL);
            instances.put(schemaURL.toString(), validator);
            return validator;
        }
    }
}

```

As you can see, the constructor is made private. Application coders will instead call the static `getInstance()` method and supply the schema to use for constraints. If an instance tied to that schema exists, it is returned, and no instantiation occurs. If, however, no instances exist, or no instances exist for the supplied schema, a new instance is created. You can see that a comment is a placeholder for a method, in the constructor, that will cause parsing of the schema to occur and a list of constraints to be built up. That prepares the instance for use. I'll look at the actual parsing, which the `SchemaParser` class will perform, a little later.

Finally, you must provide a method that allows developers to validate their data (remember, that was the point of that whole exercise!). That is also simple, and you'll skeleton out the method here. In the next article, you'll fill in the logic:

```

/**
 * <p>
 * This will validate a data value (in <code>String</code> format)
against a
 * specific constraint, and return <code>true</code> if this value is
valid
 * for the constraint.
 * </p>
 *
 * @param constraintName the identifier in the constraints to validate
this data against.
 * @param data <code>String</code> data to validate.
 * @return <code>boolean</code> - whether the data is valid or not.
 */
public boolean isValid(String constraintName, String data) {
    // Validate against the correct constraint
    // This will be coded in Article 2

    // For now, return true
    return true;
}

```

With that, I will stop here. Now I'll take a look at where you are, and where you need to go next.

Summary

Well, you've come quite a ways since our conceptual beginnings. Details about how you are going to build the framework have been solidified, and you know the classes you will need to code. One of those, the `Constraint` class, is complete, and the second, the `Validator` class, has a working skeleton and a lot of code filled in. I hate to make you wait in the middle of all that code, but that's about all the time and space I have.

In the next article, I'll look at the important task of actually parsing the XML Schema and building up a list of Constraint objects for use in the Validator. I'll also finish up the utility class, DataConverter, and complete the Validator class that was started here. Then you're all done! I'll introduce some examples so you can look at the code in action as well as discuss some advanced topics such as pattern matching and error reporting. Until then, have fun with the code, and see you online! 

About the author

Brett McLaughlin is an Enhydra strategist at Lutris Technologies who specializes in distributed systems architecture. He is the author of *Java and XML* and is involved in technologies such as Java servlets, Enterprise JavaBeans technology, XML, and business-to-business applications. With Jason Hunter, he recently founded the **JDOM** project, which provides a simple API for manipulating XML from Java applications. McLaughlin is also an active developer on the Apache Cocoon project and the EJBoss EJB server, and a cofounder of the Apache Turbine project.

Resources

- Get the source code for this article from:
<http://www.javaworld.com/jw-10-2000/validate2/Validation.zip>
- "Validation with Java and XML Schema, Part 1," Brett McLaughlin (*JavaWorld*, September 2000) where this series started:
<http://www.javaworld.com/javaworld/jw-09-2000/jw-0908-validation.html>
- "Validation with Java and XML Schema, Part 3," Brett McLaughlin (*JavaWorld*, November 2000):
<http://www.javaworld.com/javaworld/jw-11-2000/jw-1110-validation3.html>
- Curious about JDOM? Get into it with these *JavaWorld* articles:
 - "Easy Java/XML Integration with JDOM, Part 1," Jason Hunter and Brett McLaughlin (*JavaWorld*, May 2000): <http://www.javaworld.com/javaworld/jw-05-2000/jw-0518-jdom.html>
 - "Easy Java/XML Integration with JDOM, Part 2," Jason Hunter and Brett McLaughlin (*JavaWorld*, July 2000): <http://www.javaworld.com/javaworld/jw-07-2000/jw-0728-jdom2.html>
- Get JDOM today from its Website:
<http://www.jdom.org/>
- Want to know more about XML Schema? Check out the XML Schema homepage:
<http://www.w3.org/XML/Schema.html>
- Find out more about the Enhydra application server:
<http://www.enhydra.org/>
- "Objects, Objects Everywhere," Brett McLaughlin (IBM DeveloperWorks, July 2000) a data-binding series:
<http://www-4.ibm.com/software/developer/library/data-binding1/index.html?dwzone=xml>
- *Java and XML*, Brett McLaughlin (O'Reilly, 2000) pick Brett's brain on all things XML by reading his book:
<http://www.oreilly.com/catalog/javaxml>
- Discuss Java and XML in *JavaWorld's* Java and XML forum, moderated by Mark Johnson:
<http://forums.idg.net/webx?50@@.ee6b78f>
- For more XML-related articles, visit the *JavaWorld* Topical Index:
<http://www.javaworld.com/javaworld/topicalindex/jw-ti-javaxml.html>



[HOME](#) | [FEATURED TUTORIALS](#) | [COLUMNS](#) | [NEWS & REVIEWS](#) | [FORUM](#) | [JW RESOURCES](#) | [ABOUT JW](#) | [FEEDBACK](#)

Copyright © 2002 JavaWorld.com, an IDG Communications company

